CourtSy — technická dokumentace

Semestrální práce KTWA2 · Daniel Svoboda · říjen 2026 · `github.com/denysovski/ModernityRay` · `denysovski.github.io/ModernityRay/`

1. Framework a struktura projektu

React 18 s **Vite 5**. React kvůli komponentovému modelu a kvůli hooku `useSyncExternalStore`, který řeší přesně to, co v projektu potřebuji — napojení Reactu na stav uložený mimo něj. Vite kvůli rychlosti a bezkonfiguračnímu dělení kódu podle rout.

Projekt je členěný podle role, ne podle typu souboru: `src/views/` (devět obrazovek), `src/layouts/RootLayout.jsx` (společný rám), `src/components/` (`layout/`, `sections/`, `commerce/`, `ui/`), `src/lib/` (logika bez Reactu — `store.js`, `validation.js`, `format.js`), `src/data/` (veškerý obsah) a `src/routes.js` (jediné místo s definicí URL). Oddělení `lib/` je záměrné: nic v něm neimportuje React, takže se to testuje bez vykreslování.

TypeScript jsem nepoužil — zadání ho doporučuje, ale nevyžaduje, a převod existujícího JS projektu by nepřinesl body navíc. Datové struktury a veřejné funkce jsou otypované přes **JSDoc** (`@typedef Tier`, `ClubEvent`, `Court`), což dává stejné našeptávání v editoru.

---

2. Routování a architektura komponent

React Router 6 v režimu *data routeru* (`createBrowserRouter`), zvolený místo `<BrowserRouter>` kvůli třem věcem, které jinak nejsou k dispozici: `ScrollRestoration`, chybová hranice na úrovni routy (`errorElement`) a vlastnost `lazy` pro dělení kódu.

Aplikace má **devět obrazovek** — úvod, přehled a detail členství, přehled a detail areálu, kalendář a detail akce, pokladna, účet — plus stránku 404. Dvojice přehled/detail se opakuje třikrát; stejný vzor nad různými daty drží kód předvídatelný.

Strom má tři vrstvy. `RootLayout` je namountovaný jednou nad `<Outlet />`, takže se navbar, košík ani patička při navigaci neodmountují — to je smysl SPA a důvod, proč košík přežije přechod mezi routami. **Views** si berou svůj výřez dat a vlastní lokální stav (filtry, hledání). **Sdílené komponenty** jsou `PageHeader`, `Field` (formulářové pole včetně přístupnosti), `SmartLink` (rozliší routu od kotvy a od externího odkazu) a `Img` (responzivní obrázek). Filtrování a řazení se počítají přes `useMemo` ze zdrojového pole, nejsou to kopie dat ve state.

---

3. Správa stavu a práce s daty

Stav je rozdělený podle toho, jestli má přežít reload.

Trvalý stav (košík, profil, objednávky, nastavení) žije v `src/lib/store.js` mimo React a ukládá se do `localStorage`; React se na něj napojuje přes `useSyncExternalStore` uvnitř `StoreContext`. Důvody: stav je čitelný ještě před namountováním Reactu, takže počet položek v košíku je správný hned při prvním vykreslení; `useSyncExternalStore` je hook přímo určený pro externí store a na rozdíl od `useEffect` + `useState` nemůže při souběžném renderování rozpadnout konzistenci; a práce s úložištěm je celá na jednom místě.

Redux jsem nepoužil záměrně — celý stav je jeden košík, jeden profil a tři přepínače, bez asynchronních operací. Context nad externím storem dává stejnou záruku jediného zdroje pravdy bez dalších patnácti kilobajtů a zadání vlastní řešení výslovně připouští. Rozhraním je hook `useStore()`, takže výměna implementace by se nedotkla ani jedné komponenty.

Dočasný stav (otevřený košík, krok formuláře, filtry) je v `useState` tam, kde se používá. Úložiště je ošetřené proti realitě: každé čtení je v `try/catch` (privátní režim, plná kvóta), uložená data se při načtení validují (ručně upravená položka košíku se zahodí místo rozbití součtů) a klíče jsou verzované — při změně schématu se stará data smažou, místo aby na nich aplikace spadla.

---

4. Validace vstupů a zabezpečení

`src/lib/validation.js` obsahuje čisté funkce „hodnota dovnitř, chybová hláška nebo `null` ven" pro e-mail, jméno, heslo, telefon, adresu, PSČ (podle zvolené země), číslo karty, platnost a CVC. Číslo karty se kontroluje **Luhnovým**

algoritmem — stejnou kontrolou, jakou dělá každý reálný platební formulář — a z čísla se zároveň pozná vydavatel, což určuje délku CVC (Amex čtyři, ostatní tři). Pokladna validuje po krocích; chyby se neukládají do state, ale odvozují při každém renderu, takže pole se opraví ve chvíli, kdy je hodnota platná. Neplatné pole dostane `aria-invalid`, hlášku s `role="alert"` a focus skočí na první chybu.

Frontend nemůže nahradit kontrolu na serveru a netváří se tak. Dělá tři věci, které skutečně patří klientovi: (1) **nic tajného neukládá** — heslo se zkontroluje a zahodí, z karty zůstanou jen poslední čtyři číslice a vydavatel; (2) **sanitizuje před uložením** — `sanitizeText()` omezí délku a odstraní řídicí znaky, aby nešlo nafouknout úložiště ani propašovat terminátory do uloženého JSONu; (3) **odmítne nemožný vstup** — neplatný kontrolní součet nebo expirace v minulosti se do stavu vůbec nedostanou. Nikde v aplikaci není `dangerouslySetInnerHTML`; React escapuje text sám.

---

5. Testování

Vrstva	Nástroj	Počet	Co ověřuje
Unit	Vitest	87	validace, store a součty, formátování dat
Komponenty	Vitest + React Testing Library	14	přístupnost formulářového pole, průchod pokladnou
E2E	Playwright (Chromium)	12	routování, nákupní cesta, nastavení, 404

Unit testy míří na `lib/`, kde je logika, která se může tiše pokazit: hraniční data v kalendáři, Luhnův algoritmus, PSČ podle zemí, chování storu nad poškozeným `localStorage`. Testy nikdy nečtou systémový čas — funkce, které potřebují „teď", ho dostávají jako parametr, takže budou platit i příští rok. Komponentové testy ovládají pokladnu jako uživatel (`userEvent`). E2E testy běží proti **produkčnímu buildu**, ne proti vývojovému serveru — jen tak se odhalí chyba, která vznikne až rozdělením kódu na chunky. Nejdelší test projde celou cestu: členství, rezervace kurtu, čtyři kroky pokladny včetně zamítnutých vstupů, platba a nalezení objednávky v účtu po reloadu.

---

6. Profilování a optimalizace

Lighthouse nad produkčním buildem, mobilní emulace. Reporty jsou v `docs/lighthouse/`, měření se opakuje příkazem `npm run audit`.

Routa	Výkon	Přístupnost	LCP	CLS
`/`	70 → **89**	90 → **100**	9,8 s → **3,4 s**	0 → **0**
`/memberships`	68 → **92**	94 → **100**	3,2 s → **3,0 s**	0,58 → **0**
`/courts/centre-court`	57 → **91**	96 → **100**	5,0 s → **3,2 s**	0,59 → **0**

Best Practices i SEO jsou po úpravách na **100** na všech měřených routách.

- **CLS 0,58.** Dělení kódu přes `React.lazy` + `<Suspense>` vykreslilo nejdřív rám se zástupným prvkem a teprve pak obsah, čímž posunulo patičku přes půl stránky. Řešením byl přechod na `lazy` na úrovni routy — router počká na modul a vykreslí stránku najednou. Stejné chunky, stejné bajty, nulový posun.
- **LCP 9,8 s.** Fotografie byly JPEG o 1600 px a 200–550 kB. `scripts/responsive-images.mjs` z nich přes Sharp generuje WebP ve čtyřech šířkách a komponenta `Img` staví `<picture>` se `srcset`. Hlavní obrázek úvodní stránky klesl z 293 kB na 72 kB a dostal `fetchpriority="high"`.
- **Render-blocking požadavek.** Font Inter se tahal z `fonts.googleapis.com`; teď je součástí buildu (`@fontsource-variable/inter`).
- **Přístupnost.** Kontrast drobných textů nesplňoval WCAG AA, pořadí nadpisů přeskakovalo z `h2` na `h4`, tečky karuselu měly 8 px místo minimálních 24 px a sbalené mobilní menu nechávalo ~20 odkazů v pořadí tabulátoru. Opraveno. `MotionConfig` navíc respektuje systémové „reduce motion" a uživatel si ho může vynutit v nastavení účtu.

Dělení kódu zůstalo, i když u prvního načtení mnoho bajtů neušetří: sdílené chunky (`react`, `framer-motion`) se mezi nasazeními nemění, takže vracejícímu se návštěvníkovi zůstanou v cache, a jednotlivé obrazovky mají po 1–10 kB.